

深度调研系统 对标分析

以 langchain-ai/open_deep_research 为对象，对照 GPT Researcher、DeerFlow 2.0、STORM 等开源同类系统，评析优胜点与不足，给出架构调整建议与分阶段改进路线图。

目录

一、对标背景与对象	2
二、相对同类系统的优胜点	2
2.1 配置化与可移植性：最强的一项	2
2.2 上下文成本控制：架构级优势	3
2.3 澄清式人机协作内建于图	3
2.4 平台工程与可评估性	3
2.5 本轮扩展后的完整闭环	3
三、相对同类系统的不足点	3
3.1 无记忆系统（对比 DeerFlow）	3
3.2 无代码执行与数据产物（对比 DeerFlow）	4
3.3 引用可信度无校验（对比 STORM / GPT Researcher）	4
3.4 搜索生态窄（对比 GPT Researcher）	4
3.5 长任务体验短板	4
3.6 无内置成本计量与配额	4
3.7 工程质量债	4
3.8 交付形态依赖生态	5
四、架构调整建议	5
五、弥补不足与体验强化路线图	6
5.1 P0：止血与计量（1-2 周）	6
5.2 P1：能力补齐（2-4 周）	6
5.3 P2：体验跃迁（4-8 周）	6
5.4 优先级逻辑	6
六、结论	6

一、对标背景与对象

本报告以 langchain-ai/open_deep_research（下称 ODR，v0.0.16）为分析对象，选取 GitHub 上活跃度与代表性最高的四类同类深度调研（Deep Research）系统作为参照系：GPT Researcher、字节跳动 DeerFlow 2.0、斯坦福 STORM，以及 HuggingFace open-deep-research。同时以 OpenAI / Gemini 的闭源 Deep Research 产品作为能力上限参照。对比维度覆盖：定位与架构、上下文与成本控制、引用可信度、交付形态（UI 与产物）、扩展生态（搜索源、MCP、记忆、沙箱）与工程质量。

系统	定位	技术栈	关键差异点
ODR（本系统）	可配置的深度调研智能体	Python + LangGraph	BYO 模型/搜索/MCP；主管-研究员分层
GPT Researcher	可嵌入的自主调研组件	Python（自研编排）	递归式深度调研；多格式报告导出；自带 UI
DeerFlow 2.0	超级智能体工作台	LangGraph + Next.js	子智能体编排；记忆；Docker 沙箱；PPT/代码生成
STORM	维基百科式长文写作	Python（dspace 系）	大纲驱动；多视角提问；强引用规范
HF open-deep-research	轻量搜索智能体	smolagents	最小实现，代码智能体路线

表 1 对标系统概览（数据来源：各项目 GitHub 仓库与官方文档，2026-09 检索）

选择理由：GPT Researcher（约 29k stars）是社区规模与报告工程化程度的标杆；DeerFlow 2.0 与 ODR 同样构建在 LangGraph 之上，是最直接的架构对照组；STORM 代表学术界对“引用可信度”的最严格要求；HF 版本则代表极简派。四者恰好覆盖了深度调研系统的四种典型取向。

二、相对同类系统的优胜点

核心 2100 行 / 双层多智能体

ODR 以极小的核心代码量实现了与数万行级竞品同阶的调研能力

2.1 配置化与可移植性：最强的一项

ODR 通过 `init_chat_model` 以字符串配置接入任意模型供应商，搜索引擎支持 Tavily、OpenAI 与 Anthropic 原生联网搜索三种内置方案，并原生支持 MCP 外部工具协议（`streamable-http`）。“自带模型、自带搜索、自带工具”的开放度在五个系统中最高：GPT Researcher 绑定自研编排栈，DeerFlow 虽也基于 LangGraph 但组件耦合更深，STORM 的模型接入层则相对陈旧。

2.2 上下文成本控制：架构级优势

ODR 的主管-研究员分层 + 压缩回传设计（compressed_research 回传主管，原始资料永不进入主管上下文）使主管侧上下文成本随研究规模近似恒定。对照 GPT Researcher 的递归深度调研——其 breadth×depth 展开后上下文随规模线性膨胀——ODR 的设计在长任务成本上结构性占优。并行子研究员（asyncio.gather，默认 5 路）还带来墙钟时间优势，这一点 DeerFlow 与 GPT Researcher 亦有，但 ODR 的并发上限由配置直接约束、成本可预估性更好。

2.3 澄清式人机协作内建于图

ODR 将“先澄清、后调研”作为图的第一节点（clarify_with_user），以结构化输出判断是否需要追问，并在提示词层面防止循环追问。GPT Researcher 默认不澄清直接开工；DeerFlow 的人机交互依赖运行时打断；STORM 面向给定主题不承担交互。对模糊需求场景，ODR 的产出命中率更稳定。

2.4 平台工程与可评估性

ODR 是唯一自带 LLM-as-judge 评估 harness 并公开排行榜成绩的对照项目（Deep Research Bench, README 自述 RACE 0.4344, 2025 年 8 月时点第六）；同时借 LangGraph Platform / Open Agent Platform 获得免开发的多租户部署、鉴权与配置 UI 元数据（x_oap_ui_config）。竞品中仅 DeerFlow 提供同等量级的部署形态，GPT Researcher 与 STORM 的多租户能力均需自建。

2.5 本轮扩展后的完整闭环

在继承上游的基础上，本仓库已补齐自研 FastAPI 服务层（SSE 流式）、Postgres 会话持久化（AsyncPostgresSaver 检查点）与 React 前端，形成“提问-澄清-流式报告-历史回看”的产品闭环，且核心研究代码零改动。这使得下文针对不足的改造可以完全在外围层推进。

三、相对同类系统的不足点

以下不足按“对最终调研质量与产品竞争力的影响”排序，前四项与竞品存在代际差距，后四项属于工程成熟度问题。

3.1 无记忆系统（对比 DeerFlow）

会话间零记忆：用户偏好、领域背景、历史报告均不参与新会话。DeerFlow 2.0 已内置持久记忆与技能体系，支持长周期任务的上下文积累。ODR 的 LangGraph Store 依赖点（MCP OAuth 令牌存取，utils.py:293-383）已预留接入面，但当前未启用。

3.2 无代码执行与数据产物（对比 DeerFlow）

调研结果只能输出 Markdown 文本，无图表生成、无 PPT 导出、无代码运行。DeerFlow 可在 Docker 沙箱中执行 Python 完成数据分析并产出幻灯片。对于“调研 + 分析 + 汇报”的完整 workflow，ODR 目前止步于第一步。

3.3 引用可信度无校验（对比 STORM / GPT Researcher）

ODR 的引用完全由提示词驱动（压缩提示与报告提示要求编号引用与 Sources 段），无 URL 级可达性校验、无“结论-来源”对齐检查。STORM 以引用规范为一等公民，GPT Researcher 内置引用与一致性检查。在“报告可信”这一深度调研的核心卖点上，ODR 目前依赖模型自觉，是最大的质量风险点。

3.4 搜索生态窄（对比 GPT Researcher）

当前主包实际可用的检索面只有 Tavily 与两家原生搜索；legacy 目录中的 arXiv、PubMed、Exa、DuckDuckGo 等多引擎实现未接入主包，也没有本地文档 RAG。GPT Researcher 支持多搜索引擎、网页爬取与本地文档混合检索，对“私有知识 + 公网信息”混合调研场景覆盖不足是 ODR 的硬伤。

3.5 长任务体验短板

一次研究耗时数分钟且不可后台化：SSE 断开即中断本次运行（状态虽已持久化，但缺少任务队列、完成通知与断点续跑）；上下文超限依赖启发式截断（4 字符/token 近似 + 10% 递减）。对照 OpenAI/Gemini 闭源产品的“后台任务 + 完成推送”体验，差距明显。

3.6 无内置成本计量与配额

token 用量零记录，成本不可按用户/会话归因，也没有配额护栏。一次完整研究成本可达数元人民币，对外开放前这是必须补齐的工程缺口（GPT Researcher 提供 usage 追踪，DeerFlow 有任务级日志）。

3.7 工程质量债

问题	位置	影响
异常吞噬：token 检查后接 or True，任何异常静默终止研究阶段	deep_researcher.py:334	失败被伪装成低质量报告
核心包零单元测试、CI 空白（仅评估 harness）	tests/ 目录	提示词与路由改动无回归安全网
MODEL_TOKEN_LIMITS 子串匹配误命中、需手工维护	utils.py:831-846	截断逻辑对新模型失效
可观测性弱：图节点零日志，依赖 LangSmith	全仓库仅 4 处日志	线上排障困难

表 2 工程质量债清单（位置均指 ODR 源码）

3.8 交付形态依赖生态

官方交付形态是 LangGraph Studio（面向开发者调试），终端用户 UI 缺位——DeerFlow（Next.js 界面）与 GPT Researcher 均自带产品级前端。本仓库已通过自研 FastAPI + React 前端补齐，但上游同步更新时需要自行维护这层差异。

四、架构调整建议

总原则：保留内核、加固外围。ODR 的 supervisor-compression 内核是被对标验证过的优势设计，不应推倒；所有调整集中在记忆、工具、验证、任务与计量五个外围层。

调整后的目标分层如下：表现层（Web/API）之下依次为任务与计量层、验证与后处理层、记忆层、工具与检索层，内核研究图保持稳定。各层之间以接口与事件解耦，避免重蹈竞品“平台化后核心僵化”的覆辙。

调整项	现状	目标形态	对标来源
记忆层	会话间零记忆，Store 未挂载	Postgres Store 按 user 命名空间存偏好/摘要/MCP 令牌	DeerFlow memory
检索抽象	搜索 API 硬编码为 4 种枚举	Retriever 插件接口 + 本地文档 RAG 混合检索	GPT Researcher
引用校验	纯提示词约束	报告后置校验节点：URL 可达性 + 摘要-来源对齐打分	STORM
任务模型	SSE 同步阻塞、断线即断	run 后台化 + 状态机 + 完成通知（检查点已支持恢复）	闭源产品体验
产物扩展	仅 Markdown 报告	受限沙箱出图表 + PPT/DOCX 导出	DeerFlow / GPT-R
计量配额	零记录	SSE 流内累加 usage_metadata，落 usage_events/quotas 表	GPT-R usage

表 3 六项架构调整建议总览

其中“检索抽象”建议将 SearchAPI 枚举（configuration.py:11）重构为注册制 Retriever 接口：每个 Retriever 声明名称、并发度与返回格式，legacy 目录中的 arXiv/PubMed/Exa 实现可直接改造入驻，本地文档检索作为第五个内置 Retriever 接入，工作量可控。

“引用校验”建议实现为 final_report_generation 之后的独立节点：抽取报告中的全部 URL，异步做可达性探测与内容比对（报告论断与来源摘要的语义一致性），低置信引用在报告中降级标注。该节点只在最终报告阶段运行一次，成本约为整次研究的 3-5%。

五、弥补不足与体验强化路线图

5.1 P0：止血与计量（1-2 周）

事项	做法	收益
修复 or True 异常吞噬	deep_researcher.py:334 改为仅捕获 token 超限后结束	失败可见、可归因
单元测试 + CI	fake 模型驱动节点路由测试；ruff/mypy/pytest 进 GitHub Actions	改动有安全网
token 计量	SSE 循环累加 usage_metadata，落 usage_events 表	成本可归因
配额护栏	quotas 表 + 发起前校验，超限 429	可对外开放
引用校验节点	URL 可达性探测 + 低置信降级标注	报告可信度

表 4 P0 事项（全部为外围改动，核心图仅一处修复）

5.2 P1：能力补齐（2-4 周）

① 挂载 Postgres Store，启用用户级记忆（会话摘要、偏好、MCP 令牌），并在研究员系统提示中注入用户画像摘要；② Retriever 插件化，接入 arXiv/PubMed 与本地文档 RAG；③ run 后台化：任务队列 + 前端进度轮询 + 完成通知，SSE 降级为可选通道；④ 前端补配置面板（复用 x_oap_ui_config 元数据自动生成表单）与报告导出（PDF/DOCX）。

5.3 P2：体验跃迁（4-8 周）

① 受限代码沙箱（容器 + 白名单库）支持图表生成，报告内嵌数据可视化；② PPT 产物生成（调研结果到幻灯片的模板化转换）；③ 评估自动化：将 Deep Research Bench 小样本评测纳入夜间 CI，提示词改动自动对比质量与成本曲线；④ 多语言报告与引用格式化（GB/T 7714、APA 可选）。

5.4 优先级逻辑

P0 解决“能不能信、能不能开”；P1 解决“记忆与生态”，缩小与 DeerFlow/GPT Researcher 的能力代差；P2 面向“闭源产品级体验”。全程遵循本轮已验证的扩展模式：外围新增、内核少动、每个 P 阶段以评估脚本验证报告质量不后退。

六、结论

在与 GitHub 同类系统的横向对比中，ODR 的差异化优势清晰且难以复制：最强的配置化与 MCP 生态、架构级的上下文成本控制、内建的澄清式交互与公开可复现的评测成绩。其不足集中在“产品完整度”而非“架构正确性”：记忆、沙箱、引用校验、成本计量与后台任务均为可增量补齐的外围

能力。建议按 P0-P1-P2 路线推进，优先补齐计量与引用校验两项信任基础，再以检索插件化与记忆系统追赶竞品能力面，最终以代码沙箱与多产物输出完成向“调研工作台”的跃迁。

参考资料：langchain-ai/open_deep_research (GitHub)、assafelovic/gpt-researcher (GitHub)、bytedance/deer-flow (GitHub 与 deerflow.tech)、stanford-oval/storm (GitHub)、huggingface.co/blog/open-deep-research、digitalapplied.com《Four Open-Source Deep Research Agents, Tested Honestly》(2026)。检索时间 2026 年 9 月。